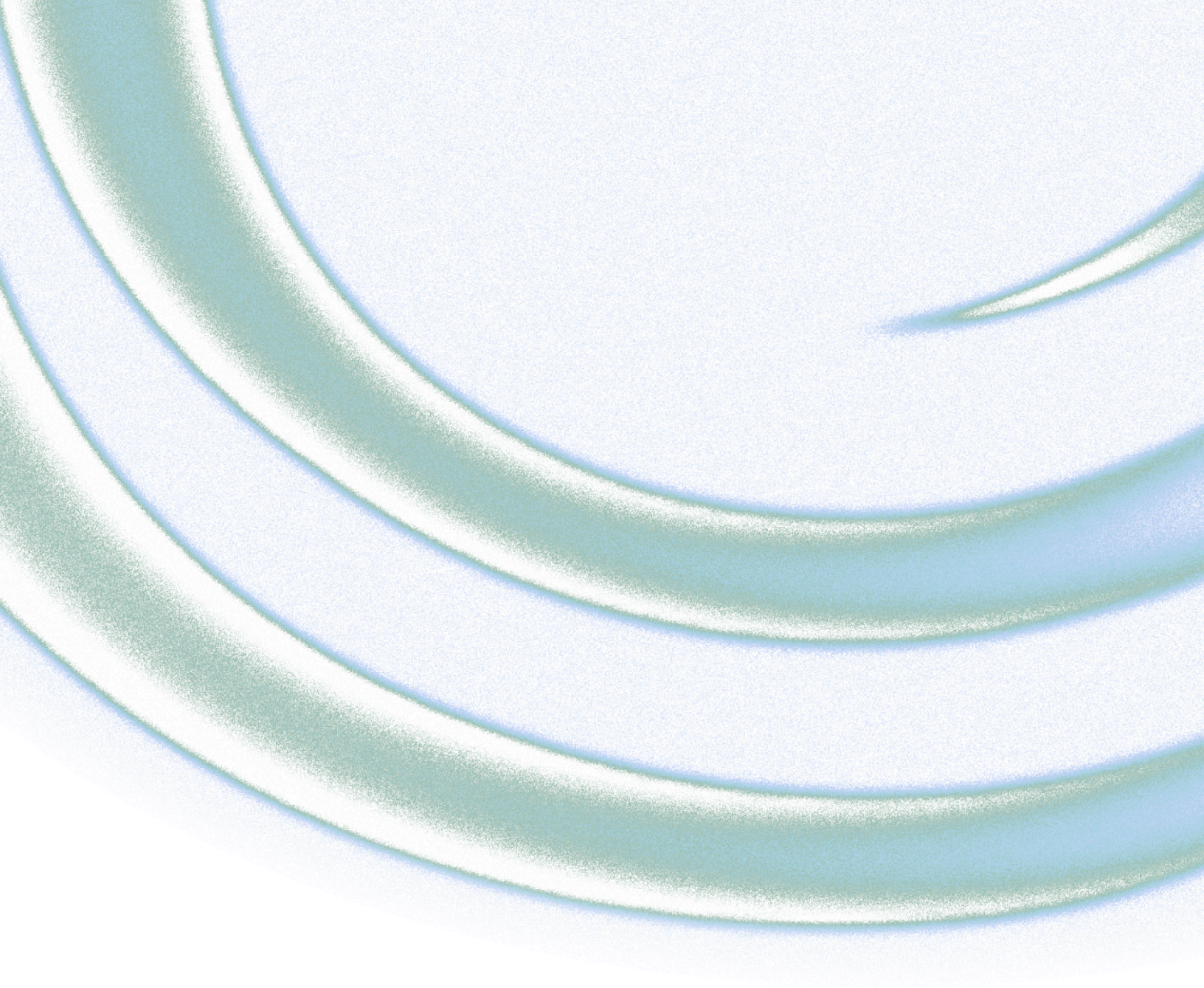


OtterSec

Stellar OFT / Sac Manager

Security Assessment

Prepared for
LayerZero

Prepared by
Otter Audits, LLC

Prepared on
September 18th, 2026

Renato Eugenio Maria Marziano

renato@osec.io

Table of Contents

1. Executive Summary 2
1.1. Overview 2
1.2. Key Findings 2
2. Scope 3
2.1. Contract scoping 3
3. Findings 4
3.1. Findings Summary 4
4. Suggestions 5
OS-LSO-SUG-00 Upgrader and Owner Share Equivalent Privileges 6
A. Vulnerability Rating Scale 7
B. Procedure 8

© 2026 Otter Audits LLC. All Rights Reserved.

1. Executive Summary

1.1. Overview

LayerZero engaged OtterSec to conduct a security assessment of the `upgradeability` implementation. This assessment was conducted on September 17th, 2026. For information on our auditing methodology, refer to Appendix B.

1.2. Key Findings

We produced 1 finding throughout this audit engagement.

In particular, the Upgrader and owner have equivalent effective privilege, where the Upgrader can replace ownership controls through new bytecode, while the owner can revoke the Upgrader role. Both keys therefore require the same level of protection and oversight (OS-LSO-SUG-00).

© 2026 Otter Audits LLC. All Rights Reserved.

2. Scope

The upgradeability implementation allows an authorized Upgrader role to replace a contract's WASM bytecode and optionally perform state migration.

The source code was delivered to us in a Git repository at `https://github.com/LayerZero-Labs/monorepo-external`.

The audit was performed against commit acf921b.

2.1. Contract scoping

Below is the list of all contracts that were within the scope:

1. `apps/oft-app/contracts/stellar/oft`
2. `apps/oft-app/contracts/stellar/oft-core`
2. `apps/oft-app/contracts/stellar/sac-manager`

© 2026 Otter Audits LLC. All Rights Reserved.

3. Findings

Findings were rated using the criteria in Appendix A.

We split the findings into **advisories** and **suggestions**. Advisories have an immediate impact and should be remediated as soon as possible. Suggestions do not have an immediate impact but will aid in mitigating future vulnerabilities.

Items are tracked with a stable identifier: `OS-LSO-ADV-*` for advisories and `OS-LSO-SUG-*` for suggestions.

3.1. Findings Summary

Severity	Count
Critical	0
High	0
Medium	0
Low	0
Informational	1

© 2026 Otter Audits LLC. All Rights Reserved.

4. Suggestions

Here, we present a discussion of the 1 suggestion we identified during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Severity	Status	Description
OS-LSO-SUG-00	Informational	× To-do	Upgrader and Owner Share Equivalent Privileges

© 2026 Otter Audits LLC. All Rights Reserved.

Upgrader and Owner Share Equivalent Privileges

ID	Severity	Status
OS-LSO-SUG-00	Informational	To-do

Description

In `OFT` and `SACManager`, the `Upgrader` role and contract owner effectively operate at the same trust level despite having different explicit permissions. The `Upgrader` can deploy arbitrary bytecode that bypasses owner authorization, replaces the owner, or removes ownership controls entirely. Conversely, the owner can revoke the `Upgrader` role before an upgrade is executed, eliminating its authority.

contracts/common/utils/stellar/common-utils/src/ownable.rs

```
181  fn renounce_ownership(env: &soroban_sdk::Env) {
182    let old_owner = enforce_owner_auth::<Self>(env);
183    assert_no_pending_transfer::<Self>(env);
184
185    OwnableStorage::remove_owner(env);
186    OwnershipRenounced { old_owner }.publish(env);
187  }
```

Remediation

Protect the `Upgrader` key and `Owner` key with equivalent access controls.

© 2026 Otter Audits LLC. All Rights Reserved.

Appendix A.

Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the general findings.

Severity	Description
Critical	Vulnerabilities that immediately result in a loss of availability, consensus divergence, or exploitable memory corruption with minimal preconditions.
High	Vulnerabilities that may result in consensus divergence or denial of service but require specific transaction or network conditions to exploit.
Medium	Vulnerabilities that may result in incorrect behavior or state divergence under specific configurations or edge cases.
Low	Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.
Informational	Best practices to mitigate future security risks. These are classified as general findings.

Resolution status is one of ✓ Resolved, ○ Ack'd, or × To-do.

© 2026 Otter Audits LLC. All Rights Reserved.

Appendix B.

Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

The design review asks whether the programs's architecture and operating assumptions support its intended behavior. We study the inputs it accepts, the decisions it makes, the information shared between components, and the services or participants it trusts. We then consider how it behaves when those inputs are unexpected or conflicting. The goal is to find conditions that can produce an incorrect decision, inconsistent state, or loss of service.

Design problems often arise from interactions rather than one defective function. Two components may each process a value correctly but interpret it differently. A failure may also leave one component ready to continue while another treats the same operation as incomplete. Local checks can look correct in both cases, so we follow the complete operation across component boundaries.

The implementation review asks whether the code matches the design. We trace untrusted data from its entry point to the point where it affects behavior, checking the assumptions made along the way. The review covers validation, error handling, state updates, concurrency, persistence, resource use, and cleanup. Its exact focus depends on the target's language, runtime, and architecture rather than a fixed list of bug classes.

The boundary between design and implementation is not exact. Missing validation may be a local coding error, while the same omission across several components may reflect an unclear rule. We classify a finding by its root cause, but judge its impact from the resulting behavior and the conditions needed to trigger it.

We begin by reading the target as a team and agreeing on its main components, trust boundaries, and expected behavior. Auditors then review different paths through the system and compare notes where those paths meet. We reproduce suspected issues with focused tests or traces when the setup allows. Before reporting a finding, another auditor reviews its trigger, impact, and proposed remediation.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process

© 2026 Otter Audits LLC. All Rights Reserved.